Cisco Confidential

Nombre: Emiliano Sánchez Juárez Valeria Nohemí Cisneros Moreno	**Matrícula**: 2169669 2178081
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: DISEÑO ARQUITECTURAL DEL SISTEMA
Fecha: 16/10/2026	
Bibliografía (Formato APA)	

ACTIVIDAD PRACTICA

Diseño arquitectural del sistema con IA

Guía de trabajo para alumnos

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

Empresa asignada	PetMarket
Integrantes	Emiliano Sánchez Juárez2169669 Valeria Nohemí Cisneros Moreno2178081
Fecha de entrega	16/12/2026

Cisco Confidential

6 Etapa 1 Preparar el caso

Seleccionen de tres a cinco casos de uso relacionados con el proceso principal. Deben incluir al menos un flujo normal y un flujo con excepción o rechazo. Para cada caso de uso, identifiquen el actor, el objetivo, la información de entrada, las reglas involucradas y el resultado esperado.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Iniciar checkout y validar inventario	- Cliente (principal) - PetMarket Plataforma (secundario)	**RN-01:** El stock debe estar disponible en el almacén. **RN-02:** En caso que no haya stock, notificar “producto agotado” para que el cliente ajuste su carrito de compras. **RN-03:** Solo se avanza a la validación de dirección y método de entrega si el stock es suficiente.	*Normal:* Inventario confirmado y checkout exitoso. *Excepción:* El cliente es notificado del producto agotado y ajusta su carrito
CU-02	Procesar pago y confirmar pedido	- Cliente (principal) - Pagos (secundario)	**RN-04:** La dirección y el método de entrega deben confirmarse válidos antes del pago. **RN-05:** El pedido se confirma solo si el pago es aprobado. **RN-06:** Si el pago es rechazado, se notificará al cliente y se volverá a intentar.	*Normal:* Pago aprobado y pedido confirmado. *Excepción:* El cliente es notificado del pago rechazado y se vuelve a intentar.
CU-03	Preparar, empacar y despachar pedido	- Almacen (principal) - Paqueteria/Reparto (secundario)	**RN-07:** Si el pedido requiere 2 almacenes, cada uno prepara su línea. **RN-08:** Si requiere un solo almacén, este prepara el pedido completo. **RN-09:** El pedido se empaca y consolida antes del despacho. **RN-10:** Si la entrega es exitosa, el proceso finaliza como “pedido entregado”. **RN-11:** Si la entrega no es exitosa, se identificará el área de oportunidad y se empaquetara un pedido nuevo al cliente, notificándose del cambio.	*Normal:* Pedido preparado, consolidado, despachado y entregado al cliente. *Excepción:* La entrega del pedido se retrasa y es entregado al cliente en una fecha diferente.

Cisco Confidential

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-04	Cancelar pedido antes del envío	- Cliente (principal) - Atención al Cliente (secundario)	**RN-12:** La cancelación es posible en cualquier etapa, mientras sea antes del envío. **RN-13:** Al cancelar se registra la cancelación y se notifica al almacén. **RN-14:** La cancelación genera un reembolso por cancelación.	*Normal:* Cancelación registrada, almacén notificado, reembolso emitido. *Excepción:*

Antes de continuar, comprueben que los casos seleccionados aparecen en las historias de usuario o en el BPMN. Si no existe esa relación, el caso debe justificarse como corrección de alcance.

7 Etapa 2 Elegir el patron de logica de negocio

Comparen Transaction Script, Table Module y Domain Model. La decisión deberá basarse en el comportamiento del sistema: cantidad de reglas, interacción entre conceptos, cambios esperados, manejo de estados y necesidad de proteger invariantes.

Trabajo solicitado

- Construyan una tabla comparativa con criterios propios del caso.
- Incluyan ventajas, riesgos y consecuencias de cada alternativa en su empresa.
- Seleccionen un patrón principal. Pueden proponer una combinación solo si delimitan dónde se usa cada patrón.
- Relacionen la decisión con al menos tres evidencias de la Unidad 2.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Cada caso de uso es un procedimiento con sus if. Con 13 reglas (stock, pago, 1 o 2 almacenes, cancelación) se duplican condiciones entre scripts.	Agrupa las reglas por tabla (Pedidos, Pagos, Inventario), pero las reglas que cruzan tablas (pago, stock y envío) quedan sin dueño claro.	Cada regla vive en el objeto al que pertenece (Pedido, Pago, Envío), sin repetirse.	Reglas de negocio derivadas del BPMN (reglas previamente establecidas, RN-01 a RN-14)
Cambios esperados	Cambiar una regla, como agregar un tercer almacén, obliga a revisar muchos scripts. Riesgo alto de olvidar alguno.	Los cambios se localizan por tabla, pero siguen afectando varios módulos.	El cambio se concentra en una clase o política (p. ej. la asignación de almacén). Es el que mejor lo absorbe.	Glosario U2: Pedido dividido, Almacén único y Método de entrega, variantes que cambian con el negocio

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Interacción entre conceptos	Los conceptos son solo datos que el script manipula. Las relaciones Pedido, Línea, Almacén y Pago se codifican a mano.	Cada módulo trabaja su tabla y la coordinación entre módulos recae en código externo.	Las relaciones (un Pedido tiene Líneas, una Línea se surte desde un Almacén, un Pedido tiene Pagos y Envíos) se modelan directamente.	Glosario U2: Pedido dividido, Almacén, Pago, Envío y Reembolso
Estados e invariantes	El estado es un campo que cualquier script puede modificar. Nada impide un pedido "despachado" sin pago aprobado.	Se valida a nivel de registro, pero no se protege el ciclo de vida completo.	El Pedido controla sus transiciones (confirmado, preparado, despachado, entregado o cancelado) y rechaza las inválidas.	BPMN U2: gateways ¿Pago aprobado?, ¿Pedido requiere 2 almacenes?, ¿Cliente cancela antes del envío?
Criterio adicional	Mínimo costo inicial. Ideal para operaciones simples y aisladas.	Costo bajo a medio. Encaja si el sistema gira en torno a tablas y reportes.	Costo inicial más alto (diseño, mapeo a base de datos, curva de aprendizaje).	Alcance del BPMN: un proceso principal con dos almacenes

Ventajas, riesgos y consecuencias en PetMarket

Patrón	Ventajas	Riesgos	Consecuencias en la empresa
Transcription Script	Rápido de construir, fácil de entender, poco diseño previo.	Lógica duplicada, scripts largos, invariantes sin protección.	Se lanza pronto, pero cada nueva regla (promociones, nuevo almacén) encarece el mantenimiento y sube la probabilidad de errores en pedidos y reembolsos.
Table Module	Orden por tabla, buen apoyo de herramientas y reportes.	Las reglas que cruzan módulos quedan dispersas. No modela bien el flujo de un pedido en varios almacenes.	Funciona para catálogos e inventario, pero el proceso de pedido seguiría fragmentado.
Domain Model	Reglas e invariantes centralizadas, extensible, refleja el lenguaje del negocio y el BPMN.	Más tiempo inicial, requiere disciplina de diseño, riesgo de sobrediseño en partes simples.	Mayor inversión al inicio, pero menos errores costosos (envíos sin pago, cancelaciones tardías) y

Cisco Confidential

			cambios más baratos a mediano plazo.

Cisco Confidential

En base a la información ya presentada, el patrón principal a usar será el Domain Model, el cual se aplica al núcleo del proceso del pedido (Pedido, Línea de pedido, Pago, Envío, Reembolso y Almacén con su inventario). Si habrá una combinación delimitada con Transaction Script, pero solo en operaciones simples y sin reglas cruzadas, como notificaciones para el cliente o para generar consultas y reportes de solo lectura.

Los problemas que resuelve dentro del caso son las invariantes del pedido, interacción entre varios conceptos y los cambios esperados:

- Invariantes del pedido: Las condiciones que se imponen y no deben cambiar son el que el pedido no se confirma hasta aprobarse el pago (RN-05), no se despacha sin empacar y consolidar (RN-09) y no se cancela después del envío (RN-11). En este caso, el patrón más adecuado sería el Domain Model ya que controla sus propias transiciones y rechaza las inválidas.
- Interacción entre varios conceptos. El caso del pedido dividido (RN-07 y RN-08) obliga a que un mismo Pedido reparta líneas entre almacenes y luego las consolide. Es una relación entre objetos, no un procedimiento lineal, y se expresa de forma natural con un modelo de objetos.
- Cambios esperados. Una tienda en crecimiento probablemente añadirá almacenes, métodos de pago y políticas de devolución. Con Domain Model, esos cambios se concentran en pocas clases en lugar de repartirse por toda la aplicación.

En cuanto al costo, el equipo debe invertir más tiempo al inicio en diseñar el modelo y mapearlo a la base de datos, y necesita conocer el patrón. Para controlarlo, se aplica sólo al núcleo del pedido y no a todo el sistema. Si el proyecto tuviera un plazo muy corto y reglas estables, Transaction Script sería más razonable, pero este no es el caso.

Entre su relación con las evidencias de la Unidad 2, se encuentran las reglas de negocio derivadas del BPMN, ya que su cantidad y el cruce entre ellas justifican centralizarlas en el dominio. También está el BPMN en sí, con los gateways (¿pago aprobado?, ¿pedido requiere 2 almacenes?, ¿cliente cancela antes del envío?) y las ramas de excepción muestran estados y transiciones que el Pedido debe proteger. Y por último, el glosario con términos como Pedido dividido, Almacén, Pago y Reembolso son la base de las clases del modelo.

Producto de la etapa: una decisión argumentada de una a dos páginas. No basta escribir que un patrón es más fácil, moderno o escalable. La explicación debe indicar qué problema concreto del caso ayuda a resolver y qué costo introduce.

8 Etapa 3 Construir el mapa de dominio

Transformen el lenguaje y las reglas de la Unidad 2 en un mapa de responsabilidades. Identifiquen los elementos que el sistema debe reconocer, proteger o comunicar. No conviertan automáticamente cada tabla de datos en una entidad del dominio.

Elementos que deben identificar

Elemento	Pregunta que debe responder el equipo
Lenguaje ubicuo	Terminos usados por usuarios, analistas y desarrolladores con el mismo significado.

Cisco Confidential

Elemento	Pregunta que debe responder el equipo
Entidades	Conceptos con identidad y ciclo de vida que deben distinguirse aun cuando cambien sus datos.
Value Objects	Conceptos definidos por sus valores y reglas, sin identidad propia relevante.
Aggregates	Grupos que requieren un limite de consistencia y una raiz para controlar cambios.
Domain Services	Comportamientos del dominio que no pertenecen naturalmente a una sola entidad.
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.

Producto de la etapa

- Diagrama del mapa de dominio con relaciones y límites claramente identificados.
- Glosario actualizado con los términos nuevos o corregidos.
- Ficha de cada Aggregate que indique su raíz, responsabilidades, invariantes y eventos posibles.
- Explicación de las decisiones dudosas o descartadas.

Glosario actualizado

Término	Rol
Pedido dividido (split order)	Variante del Pedido cuyas líneas salen de dos almacenes. Es un estado o tipo del Pedido, no un aggregate
Almacen unico/Preparar pedido	Variante del Pedido que sale completo de un solo Almacén
Almacén	Raíz del Aggregate Almacén (dos instancias: Almacén 1 y 2).
Inventario	Conjunto de Existencias dentro de cada Almacén.
Preparar línea	Comportamiento del Almacén. Genera el evento LíneaPreparada.
Empacar y consolidar pedido	Comportamiento del Pedido. Genera PedidoConsolidado.
Confirmar pedido	Comportamiento del Pedido. Solo es válido con pago aprobado. Genera PedidoConfirmado.
Procesar pago / Pago rechazado	Comportamiento y resultado del Pago. Generan PagoAprobado o PagoRechazado.
Producto agotado	Resultado de validar el inventario. Genera ProductoAgotado.
Entrega fallida / Entrega	Resultado del Envío. Generan EntregaFallida o PedidoEntregado.

Cisco Confidential

exitosa	
Cancelación	Comportamiento del Pedido. Genera PedidoCancelado.
Reembolso por cancelación / por devolución	Los dos tipos del Aggregate Reembolso.
Método de entrega	Value Object del Pedido.
Cliente, Atención al Cliente, Paquetería / Reparto, PetMarket (Plataforma)	Actores y roles. No son Aggregates.
Stock disponible en almacén	Disponibilidad del Producto dentro del Almacen.
Carrito	Selección de productos del cliente antes de iniciar el checkout. Aún no es Pedido.
Pedido	Compra del cliente desde que inicia el checkout (estado “Pendiente de pago”) hasta que se entrega o cancela.
Línea de pedido	Producto y cantidad dentro de un Pedido, asignada a un almacén.
Existencia	Cantidad de un producto en un Almacén concreto
Pago	Intento de cobro de un Pedido, con su resultado
Pasarela de pagos	Servicio externo que autoriza la transacción (se menciona en el glosario, pero no se define).
Envío	Salida física del pedido hacia el cliente, con su resultado de entrega.
Reembolso	Devolución de dinero, con su tipo y motivo.
Reserva	Cantidad apartada de una Existencia para un Pedido, aún no despachada.

Fichas de Aggregates

	Raíz	Responsabilidades	Invariantes	Eventos
Pedido	**Raíz:** Pedido. **Contiene:** Líneas de pedido, Dirección, Método de entrega y Dinero (total).	Confirmar el pedido, repartir líneas por almacén (único o dividido), empacar y consolidar, y cancelarse.	• No se confirma sin pago aprobado. • No se despacha sin empacar y consolidar. • Un Pedido dividido no se consolida hasta que ambas líneas estén preparadas. • No se cancela después del despacho.	PedidoConfirmado, PedidoConsolidado, PedidoDespachado, PedidoEntregado, PedidoCancelado

Cisco Confidential

			• El total es la suma de sus líneas.	
Almacén	**Raíz:** Almacén. **Contiene:** Existencias con su Cantidad.	Validar disponibilidad, reservar y preparar líneas.	• La cantidad disponible nunca es negativa. • No se prepara una línea con más cantidad de la que existe.	ProductoAgotado, LíneaPreparada
Pago	**Raíz:** Pago. **Contiene**: Dinero, Método de pago y resultado.	Enviar la transacción a la pasarela, registrar aprobación o rechazo y permitir reintentos.	• El monto coincide con el total del Pedido. • Un pago aprobado no se procesa dos veces. • Solo se reintenta un pago rechazado.	PagoAprobado, PagoRechazado
Envío	**Raíz**: Envío. **Contiene:** Dirección destino y resultado de entrega.	Despachar y registrar entrega exitosa o fallida.	• Solo existe para un Pedido ya consolidado. • No pasa a “entregado” sin haber sido despachado. • Una entrega fallida deriva en devolución o gestión de Atención al Cliente.	PedidoDespachado, PedidoEntregado, EntregaFallida
Reembolso	Raíz: Reembolso. Contiene: Dinero, Motivo y Tipo (por cancelación o por devolución).	Devolver el dinero al cliente.	• Solo se reembolsa un pago aprobado. • El monto no excede lo pagado. • Un pago no se reembolsa dos veces.	ReembolsoEmitido

Decisiones descartadas

- **Pago dentro del Aggregate Pedido.** Se descartó porque el pago tiene su propio ciclo de vida: puede aprobarse, rechazarse y reintentarse varias veces, y lo ejecuta otro actor (el carril de Pagos en el BPMN). Si el Pago viviera dentro del Pedido, cada intento de cobro bloquearía todo el pedido, aunque el cobro no tenga nada que ver con las líneas ni con el envío. Por eso Pago es un Aggregate separado que se relaciona con el Pedido por su identificador. Así el Pago protege sus propias reglas (monto igual al total, un pago aprobado no se repite) sin interferir con el Pedido.

- **Inventario dentro del Pedido.** Se descartó porque muchos pedidos compiten por las mismas existencias al mismo tiempo. Si el inventario fuera parte del Pedido, el límite de consistencia sería enorme y habría bloqueos y cuellos de botella cada vez que dos clientes compren el mismo producto. La disponibilidad pertenece al Almacén, que es quien protege la regla de que la cantidad nunca puede ser negativa, y el Pedido solo guarda a qué almacén pertenece cada línea.

- **Almacén 1 y Almacén 2 como Aggregates distintos.** Se descartó porque ambos almacenes siguen exactamente las mismas reglas. Lo único que cambia entre ellos es su identidad, no su comportamiento, así que se modelan como un solo Aggregate Almacén con dos instancias. Crear dos

clases habría duplicado reglas y obligado a modificar ambas ante cualquier cambio. Esta decisión también facilita agregar un tercer almacén, algo que se anticipó en la Etapa 2.

- **Pedido dividido como Aggregate propio.** Se descartó porque dividir un pedido solo cambia cómo se reparten las líneas entre almacenes, no las reglas del pedido. Un pedido dividido sigue necesitando pago aprobado para confirmarse, sigue consolidándose antes del despacho y sigue sin poder cancelarse después de despachado. Por eso se maneja como un estado del Pedido (el atributo dividido). En la Etapa 7 se confirmó que modelarlo como subclase tampoco era buena idea, porque una subclase cambiaría la precondición de consolidar() y no podría sustituir al Pedido sin alterar su comportamiento.

- **Cliente como Aggregate.** Se descartó en esta etapa porque, en el BPMN, el cliente solo aparece como actor que inicia el proceso, decide el método de entrega y puede cancelar. El proceso no define reglas propias del cliente que haya que proteger, así que basta con referirse a él por su identificador (IdCliente). Si más adelante aparecen reglas propias (por ejemplo, cuentas, direcciones guardadas o historial), esta decisión se reconsidera.

- **Atención al Cliente y Paquetería como Aggregates.** Se descartaron porque son roles y actores del proceso, no conceptos con estado ni invariantes que proteger. Atención al Cliente dispara la cancelación o gestiona una entrega fallida, y Paquetería ejecuta la entrega, pero ninguno tiene reglas propias que el dominio deba proteger. Sus intervenciones se representan con servicios de aplicación y puertos, no con Aggregates.

- **Convertir cada tabla en una entidad.** No se aplicó, porque la actividad pide expresamente no hacerlo. Por ejemplo, “Dirección” y “Método de entrega” pueden ser tablas en la base de datos, pero en el dominio son Value Objects: se definen por sus valores y sus reglas de validez, y no tienen identidad propia que seguir en el tiempo. Tratarlos como entidades habría añadido ciclo de vida y complejidad sin ningún beneficio.

Decisiones dudosas

Además de las descartadas, tres decisiones se dejaron abiertas en lugar de descartarlas:

- **Carrito:** quedó fuera del mapa porque el BPMN no define reglas sobre él. Se reconsidera si aparecen carritos guardados o promociones.
- **Reserva de stock:** se modeló dentro de Existencia, sin Aggregate propio, porque el BPMN no muestra una reserva explícita. Se confirma si el equipo la necesita, pues implicaría liberar el stock al cancelar o rechazar el pago.
- **Devolución:** se absorbió en Envío y Reembolso porque el BPMN la muestra como una sola tarea. Se separaría si creciera, por ejemplo con inspección o reingreso a inventario.

9 Etapa 4 Diseñar la Service Layer

Diseñen los Application Services que expondrán los casos de uso. Cada método debe representar una intención completa del usuario o de otro sistema. El servicio coordina la operación: recibe la solicitud, obtiene los objetos necesarios, invoca el comportamiento del dominio, solicita la persistencia y devuelve un resultado.

La Service Layer no debe convertirse en un lugar donde se acumulen todas las reglas. Si una condición describe lo que el negocio permite, calcula o protege, el equipo debe justificar dónde vive esa regla y por qué.

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
CheckoutService	`iniciarCheckout` (CU-01)	`IniciarCheckoutCommand` (carrito, domicilio, método de entrega)	`CarritoRepository`, `InventarioAlmacenRepository`, `PedidoRepository`, `UnitOfWork`	`ResultadoCheckout` (pedido creado o lista de productos agotados)	RN-01, RN-02, RN-03
PagoService	`procesarPago` (CU-02)	`ProcesarPagoCommand` (pedido, método de pago)	`PedidoRepository`, `PasarelaPago` (puerto), `UnitOfWork`	`ResultadoPago` (aprobado o rechazado con reintento disponible)	RN-04
PreparacionPedidoService	`dividirPedido`, `marcarLineaPreparada`, `consolidarPedido` (CU-03)	`PedidoId`, `AlmacenId`, `LineaId`	`PedidoRepository`, `InventarioAlmacenRepository`, `UnitOfWork`	Estado actualizado del pedido	RN-05, RN-06
CancelacionService	`cancelarPedido` (CU-04)	`CancelarPedidoCommand` (pedido, solicitante)	`PedidoRepository`, `PasarelaPago`, `UnitOfWork`	`ResultadoCancelacion` (cancelado con reembolso, o rechazada)	RN-07, RN-08

Revision obligatoria

- ¿El nombre del método expresa una acción del negocio?

Cisco Confidential

- ¿El método coordina o contiene decisiones que deberían estar en el dominio?
- ¿El servicio depende de interfaces o de detalles concretos de base de datos?
- ¿La operación tiene un inicio y un final transaccional comprensible?
- ¿Los errores del dominio pueden explicarse con términos del negocio?

10 Etapa 5 Separar persistencia y transacciones

Definan interfaces de Repository en términos del dominio y describan, por separado, cómo podrían implementarse en infraestructura. Los métodos deben expresar lo que el dominio necesita recuperar o guardar, sin exponer consultas SQL como parte del modelo.

También indiquen qué operaciones deben completarse juntas. La propuesta de Unit of Work debe explicar dónde inicia la transacción, qué cambios incluye y qué ocurriría ante un error.

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
PedidoRepository	Recuperar y guardar el pedido completo con sus líneas	`obtenerPorId(pedidoId)`, `guardar(pedido)`	Aggregate `Pedido`
CarritoRepository	Leer el carrito al iniciar checkout	obtenerPorCliente(clienteId)	Carrito
InventarioAlmacenRepository	Conocer la existencia por producto y almacén	`existenciaDe(producto)` → existencia por almacén	Valor de existencia (no se guarda desde el dominio)

11 Etapa 6 Separar Commands y Queries

Apliquen CQRS básico para hacer explícita la diferencia entre cambiar el estado y consultar información. No es obligatorio construir bases de datos separadas. El objetivo es que cada operación tenga una responsabilidad clara y que las consultas puedan devolver un modelo apropiado para lectura.

Trabajo solicitado

- Seleccionen al menos dos operaciones que cambien el estado y descríbanlas como Commands.
- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.
- Definan el Handler de cada operación y sus dependencias.
- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.
- Justifiquen si alguna parte del caso no necesita esta separación.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	IniciarCheckoutCommand	carrito, domicilio, método de entrega	IniciarCheckoutHandler	`PedidoId` o lista de productos agotados	Cambia estado: crea el Pedido (CU-01)

Cisco Confidential

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	ProcesarPagoCommand	pedido, método de pago	ProcesarPagoHandler	aprobado, o rechazado con reintento disponible	Cambia el estado del Pago (CU-02)
Command	CancelarPedidoCommand	pedido, solicitante	CancelarPedidoHandler	cancelado con reembolso, o rechazado	Aplica RN-07 y RN-08 (CU-04)
Command	MarcarLineaPreparadaCommand	pedido, línea, almacén	MarcarLineaPreparadaHandler	estado actualizado	Cambia estado desde el almacén (US10)
Query	ConsultarEstadoDelPedido	PedidoId	ConsultarEstadoDelPedidoHandler	EstadoPedidoView	US13: el cliente consulta sin cambiar nada
Query	ConsultarCancelacionesYDevoluciones	rango de fechas	ReporteCancelacionesHandler	ReporteCancelacionesView	US18: reporte por periodo

12 Etapa 7 Revisar la propuesta con SOLID

Revisen la propuesta completa antes de cerrar los diagramas. El análisis debe mostrar decisiones concretas y no una definición memorizada de cada principio.

Principio	Pregunta de revision	Evidencia que deben buscar
S	¿Cada clase o servicio tiene una razon principal para cambiar?	Responsabilidades mezcladas, clases extensas, servicios que hacen de todo.
O	¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones?	Condicionales que deben modificarse por cada nueva politica.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado?	Subtipos que rechazan operaciones o alteran contratos.
I	¿Cada consumidor depende solo de las operaciones que utiliza?	Interfaces grandes con metodos irrelevantes para algunos clientes.
D	¿La aplicacion y el dominio dependen de abstracciones?	Dependencias directas de SQL, frameworks o servicios externos.

Registren al menos cinco hallazgos. Un hallazgo puede confirmar una buena decisión o señalar una corrección necesaria. Para cada uno, incluyan evidencia, impacto y cambio realizado.

Cisco Confidential

13 Etapa 8 Representar y documentar la arquitectura

Diagrama de clases

El diagrama debe mostrar las clases relevantes para el proceso elegido, sus responsabilidades y relaciones. Diferencien visualmente dominio, aplicación e infraestructura. Incluyan interfaces de Repository, Application Services y los elementos del dominio que hayan justificado.

Diagramas de secuencia

Construyan dos diagramas de secuencia. Uno debe representar un flujo principal y el otro un flujo con una regla, excepción o alternativa importante. Los participantes deben corresponder con los componentes del diagrama de clases.

Architecture Decision Records

Elaboren al menos tres ADR. Cada ADR documentará una decisión que tenga consecuencias para el diseño. Como mínimo, registren la selección del patrón de lógica de negocio y dos decisiones adicionales tomadas durante la actividad.

Campo del ADR	Contenido que debe redactar el equipo
Titulo	Nombre breve de la decision.
Estado	Propuesta, aceptada, reemplazada o descartada.
Contexto	Problema, restricciones y evidencia relevante de U2.
Alternativas	Opciones consideradas y sus consecuencias.
Decision	Opcion elegida y razon concreta.
Consecuencias	Beneficios, costos, riesgos y trabajo posterior.
Trazabilidad	IDs de necesidades, reglas, historias o escenarios relacionados.

Herramientas permitidas

Pueden utilizar draw.io, PlantUML, Mermaid u otra herramienta que permita entregar diagramas legibles. Los archivos fuente deben incluirse en el repositorio; no entreguen únicamente capturas de pantalla.

14 Vibe Coding con IA

Usarán IA para convertir descripciones del negocio en alternativas de arquitectura, generar estructura inicial, revisar código y producir borradores de diagramas. El equipo sigue siendo responsable de la arquitectura y debe verificar cada resultado contra los documentos de la Unidad 2.

Cisco Confidential

Ciclo de trabajo esperado

Paso	Accion	Evidencia
1	Preparar contexto	Seleccionar fragmentos relevantes del BRD, historias, reglas, BPMN y glosario.
2	Solicitar propuesta	Pedir una alternativa concreta y exigir que la IA declare supuestos.
3	Comparar	Contrastar la respuesta con el caso y con los conceptos de la unidad.
4	Corregir	Modificar nombres, responsabilidades, limites y dependencias que no correspondan.
5	Validar	Revisar trazabilidad, consistencia entre diagramas y comportamiento esperado.
6	Registrar	Guardar prompt, respuesta, decision humana y version resultante.

Prompts de apoyo

Los siguientes formatos son puntos de partida. Completen los corchetes con información de su empresa y eviten enviar datos que no formen parte del caso académico.

- Patrones: Con base en [reglas, cambios y casos de uso], compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
- Dominio: Revisa este glosario y estas reglas [contenido]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
- Service Layer: A partir de estos casos de uso [contenido], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.
- SOLID: Analiza este diseño [contenido] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
- Diagramas: Convierte esta arquitectura validada [contenido] en un borrador PlantUML o Mermaid. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

Cisco Confidential

Bitacora obligatoria de IA

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Entender la actividad y revisar la Unidad 2	Se subieron el PDF de PetMarket y la guía de la Actividad 3, y se pidió ayuda con el componente de lógica de negocio	Lista de hallazgos: orden de “Confirmar pedido” y “Procesar pago”, gateway de stock, Gherkin y matriz ausentes, reglas sin ID, portada con otro curso	Los IDs RN-01 a RN-08 los propuso la IA; no existen en U2	(Verificar los hallazgos contra el PDF y decidir cuáles registrar)
2	Entender la actividad y revisar la Unidad 2	Se pidió empezar por la tabla de casos de uso y la comparación de patrones	CU-01 a CU-04 con trazabilidad a historias; tabla comparativa que favorece Domain Model	El argumento de “cambios esperados” es supuesto, porque el BRD no define metas; la IA inclinó la decisión hacia Domain Model	(Aceptar, ajustar o cambiar el patrón, y redactar ADR-001)
3	Service Layer, Repository y Unit of Work (Etapas 4 y 5)	Se pidió desarrollar ambas etapas	Cuatro servicios, tres repositorios, Unit of Work con eventos tras el commit	Supuestos: pasarela de pago fuera de la transacción; nombres provisionales de los objetos del dominio; el BRD no dice si el inventario se reserva	(Decidir sobre la pasarela y validar la reserva de inventario con el stakeholder)
4	Mapa de dominio y CQRS (Etapas 3 y 6)	Se pidió desarrollar ambas etapas	Aggregates, Value Objects, eventos, diagrama Mermaid, fichas, Commands y Queries	Supuestos: estados del Pedido, `Pago` dentro de `Pedido`, una línea con un solo almacén, `Cliente` y `Reembolso` descartados	(Confirmar o corregir cada supuesto)

Cisco Confidential

15 Etapa 9 Validar la propuesta

Antes de entregar, recorran cada caso de uso sobre los diagramas y el diseño. El equipo debe poder explicar qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve.

Validacion	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	**Parcial.** CU-01 a CU-04 se relacionan con US02 a US11, US15 y US16, y cada servicio del diagrama de clases responde a un caso de uso. Excepciones: `EnvioService` y `EnvioRepository` (US14) no tienen caso de uso, porque no hay CU-05, y US17 (Atención al Cliente) queda fuera del diseño detallado. Hay que agregar CU-05 o justificarlos como corrección de alcance, y falta la matriz de trazabilidad formal.
Lenguaje	¿Los mismos terminos significan lo mismo en BRD, UML, codigo y ADR?	**Parcial.** Los nombres de clases, servicios y eventos coinciden entre el mapa de dominio, el diagrama de clases, las secuencias y los ADR. Pendiente: agregar al glosario `Despachado`, `Reintento de pago`, `Guía de envío`, `Aggregate` y `Domain Event`, y decidir si los identificadores del código llevan acento (`Envio`, `Almacen`) o no, y aclararlo en el glosario para que no parezcan términos distintos.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patron elegido con una justificacion?	**Cumple, con dos puntos a defender.** Las reglas de cancelación (RN-07), reintento (RN-04) y consolidación (RN-06) viven en `Pedido`, y la asignación de almacén en `AsignadorDeAlmacenes`. Los servicios solo coordinan: cargar, invocar, guardar y llamar a la pasarela. Puntos dudosos: `Pago` dentro de `Pedido` y los tres métodos de `PreparacionPedidoService`.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	**Parcial.** Los participantes y mensajes de las dos secuencias existen en el diagrama de clases (`PagoService`, `CancelacionService`, `Pedido`, `PedidoRepository`, `UnitOfWork`, `PasarelaPago` y los métodos usados). Discrepancia detectada: la secuencia 1 no llama a `confirmar()` después de `registrarPagoAprobado()`, aunque `Pedido` tiene ese método. Hay que decidir si `registrarPagoAprobado()` confirma internamente o si se agrega la llamada.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	**Cumple en el diseño.** El dominio solo conoce las interfaces de Repository; SQL, el sistema de inventario y la pasarela están en adaptadores de infraestructura. Aún no está verificado en código, porque el código base no existe todavía.

Validacion	Pregunta	Resultado del equipo
Transaccion	¿El limite de la operacion y su manejo de error estan definidos?	**Parcial.** Está definido dónde empieza y termina el Unit of Work (ADR-002 y ADR-003) y qué pasa ante un error de dominio (`rollback`). Siguen abiertos: idempotencia del cobro, falla del reembolso después del `commit`, edición simultánea del mismo pedido y el máximo de reintentos de pago.
IA	¿Cada aporte de IA fue revisado y su uso esta registrado?	**Parcial.** La bitácora tiene 5 iteraciones con objetivo, respuesta útil y supuestos. Faltan las iteraciones de los diagramas (clases, secuencias, archivos draw.io) y, sobre todo, **la decisión humana de cada una**, que es lo que evalúa la rúbrica.

20 Preguntas para la defensa

- ¿Qué evidencia del negocio influyó más en la selección del patrón?

R=El BPMN tiene seis carriles y tres finales, así que el pedido interactúa con pago, almacenes, paquetería y atención al cliente.

El BRD pide reglas explícitas para las excepciones (pago rechazado, producto agotado, pedido dividido, cancelación, entrega fallida).

Las historias US09 a US11 (pedido dividido por línea), US15 y US16 (cancelar solo antes de “despachado” y reembolsar).

Aclaren con honestidad que “cuántas veces cambiarán las reglas” no está en la Unidad 2, así que ese argumento es un supuesto y no el que sostiene la decisión.

- ¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?

R=

Candidatas de lo que trabajamos:

- La asignación de almacén a cada línea: usa existencias de varios almacenes, así que no cabe en una sola entidad. Terminó como Domain Service (`AsignadorDeAlmacenes`).
- El cobro a la pasarela: ¿dentro o fuera de la transacción? Quedó fuera (ADR-002).
- `Pago` dentro de `Pedido` o aparte: sigue como decisión dudosa.

- ¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?

R=

Cisco Confidential

- La regla más clara es RN-07: no se cancela un pedido ya `Despachado`, y vive en `Pedido.cancelar()`.
- Si estuviera en `CancelacionService`, otro servicio (por ejemplo, uno de atención al cliente o un proceso nocturno) podría llamar a `pedido.cancelar()` sin pasar por esa validación y dejar el pedido en un estado inválido. Es la diferencia entre proteger la regla en el dominio y recordar aplicarla en cada servicio.
- Con RN-04 (límite de reintentos) pasa lo mismo.

- ¿Qué cambió después de la revisión con SOLID?

R=Todavía no la hemos hecho, así que no hay nada que contar. Hagan la Etapa 7 antes de la defensa, con al menos cinco hallazgos y el cambio realizado en cada uno. Esta pregunta se responde con esos cambios, y si dicen que no cambió nada, suena a que no se revisó.

- ¿Qué propuesta de la IA rechazaron o corrigieron y por qué?

R=Esto solo lo pueden responder ustedes, y debe coincidir con la columna “Decisión del equipo” de la bitácora. Para ayudarles a recordar, estos son puntos donde mis propuestas tuvieron problemas o supuestos que ustedes deberían haber revisado:

- Los IDs de reglas RN-01 a RN-08 los propuse yo, no existen en el BRD.
- Servicios y Handlers duplicados, que hubo que unificar.
- `EnvioService` que faltaba en la primera versión.
- La flecha “usa” hacia Value Objects que se ve mal en el diagrama de clases.
- La secuencia de pago que no llama a `confirmar()`.

- ¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?

R=

- Con Domain Model aceptan más estructura y diseño inicial, más clases y más disciplina para no escribir reglas en los servicios.
- Si el negocio confirmara que las reglas casi no cambian, Transaction Script habría sido más barato.
- Una consecuencia relacionada: el modelo de lectura puede mostrar un estado desactualizado un momento (ADR-004), lo que choca con el objetivo del BRD de información actualizada.

https://github.com/emilianosanjua/DISENO_ARQUITECTURAL_DEL_SISTEMA